

Ciallo: The next-generation vector paint program

Shen Ciao
Individual Contributor
China
1500013714@pku.edu.cn

Li-Yi Wei
Adobe Research
USA
review@liyiwei.org



Figure 1: GPU-rendered brush strokes with a variety of styles in Ciallo.

ABSTRACT

We introduce Ciallo, an open-source next-generation vector paint program, which supports GPU-rendered stylized strokes and real-time bucket (region) fill. All strokes and fill regions are represented in editable polyline vector format instead of in raster format. Our novel stroke rendering methods are simple to implement and can potentially replicate the vast majority of brushes in other paint programs. The novel bucket fill can automate parts of the Japanese anime coloring process. In summary, Ciallo presents a new technical paradigm for the fundamentals of 2D digital art production.

CCS CONCEPTS

• Computing methodologies → Computer graphics.

KEYWORDS

Digital painting, vector graphics, coloring, arrangement, graphics processing unit (GPU), stylized stroke

ACM Reference Format:

Shen Ciao and Li-Yi Wei. 2023. Ciallo: The next-generation vector paint program. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH '23 Talks)*, August 06–10, 2023, Los Angeles, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3587421.3595418>

1 INTRODUCTION

Digital artists use paint programs to create their artworks. Traditional paint programs mainly use the CPU to render strokes, like Photoshop, Clip Studio Paint, and Krita, which tends to be slow and may cause a laggy drawing experience [Rutten 2022]. There are existing works about GPU-rendered standardized strokes [Dokter et al. 2019; Kilgard 2020], but the standardized strokes or paths may not satisfy artists' demands for diverse styles.

In Ciallo, strokes are represented in polylines and rendered via GPUs, as inspired by Blender Grease Pencil [Blender Online Community 2023]. We call our rendering methods *Articulated*, which can potentially render the vast majority of stylized strokes (brushes) in traditional digital paint programs. The methods are fully built within modern GPU graphics and compute pipelines, and simple to implement and free of tessellation.

In order to be compatible with our polyline vector representation, we propose a new region (bucket) fill method on vector images to replace the common flood fill method. In Ciallo, the feature is called *Vector Bucket Fill*, which uses vector lines to label the enclosed regions to fill. Users can edit those vector lines, then the regions to fill are automatically updated in real time. The feature is inspired by the standard coloring process in the Japanese anime industry. Artists responsible for drawing lines use colored strokes to indicate shadow or highlight regions. Then artists responsible for filling regions need to bucket fill in these regions manually. The vector bucket fill in Ciallo automates this process in real-time as shown in fig. 2a, fig. 2b and fig. 2c. It is built upon computational geometry tools to solve the arrangement and point-location problem.

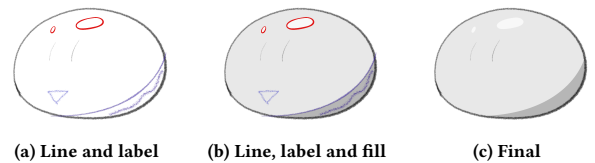


Figure 2: Automate Japanese anime coloring process via our vector bucket fill method.

In recent years, the main topic of research works on filling color is to handle contour gaps automatically [Parakkat et al. 2022; Yan et al. 2022; Yin et al. 2022]. Our vector bucket fill is complementary to most of these prior works. These works usually need flood-fill as the final step to color images after padding gaps. This process can be replaced with our vector fill method. Additionally, we do want to offer similar gap-handling features in Ciallo while keeping the method simple and effective. Therefore, we offer a feature called *Visibility* to help users detect contour gaps in their drawings, but

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH '23 Talks, August 06–10, 2023, Los Angeles, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0143-6/23/08.

<https://doi.org/10.1145/3587421.3595418>

not automatically pad them. It is directly built upon the visibility in computational geometry.

Ciallo is open-source [Cialo 2023]. For now, it still lacks critical auxiliary features for artists to work with, like layer, animation, and timeline systems. But hopefully, in the future, we can create 2D content represented in polyline vector formats to be widely used in areas like 2D video games, user interfaces, and virtual avatars.

2 METHODS

2.1 Stroke Rendering

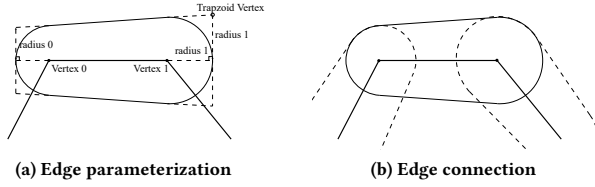


Figure 3: Vanilla stroke rendering.

Vanilla stroke. We start with rendering the vanilla stroke as shown in fig. 1a. The stroke is drawn in solid color without stylization. For each edge, we create four new vertices which form a trapezoid, as illustrated in fig. 3a. The position of each trapezoid vertex is equal to the position of the corresponding polyline vertex offset by the stroke width value, along the tangent or normal direction of the edge. The fragment shader then discards the redundant pixels in the corners. Then every single edge is naturally connected, as fig. 3b shows. It looks like the diagram of an articulated arm or gears that drive a bicycle.

Stamp stroke. In traditional paint programs, as users draw on canvas, the programs place predefined stamp textures evenly along the drawing trace, which we call the stamp method. This stamp method is very popular. All default available brushes in GIMP, more than 90% in Krita, are drawn in this method. In our method, we stamp equidistantly along the polylines, as shown in fig. 1b and fig. 1c.

For each edge of the polyline, we use the vanilla method to generate four trapezoid vertices as shown in fig. 4a. For each pixel in the trapezoid, it needs to know all the stamps within the "effect range". Stamps within this range can affect the color of the current pixel, its length is equal to the interpolated thickness. To know the stamp position, we need to know about the distance along the polyline from effect range to the very first point of stroke, which is calculated with a compute shader that does prefix sum on segments' length beforehand. For each stamp within the effect range, we get the texture coordinates of the current pixel (fig. 4b), sample color on the stamp texture then blend on the current pixel.

Airbrush stroke. Airbrush stroke is the type of stroke that has an alpha gradient. In our airbrush, given an arbitrary gradient function $A(y)$, we need to get the appropriate value of $A_{joint}(p)$. We call the disk in the overlapping area *Joint*, and non-overlapping area *Bone*. We set a Cartesian coordinate. The starting point of the

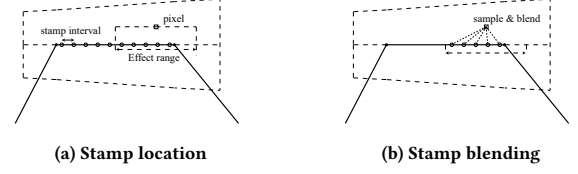


Figure 4: Stamp stroke rendering.

edge is origin, x-axis directs to the tangent. Here is the formula to implement it:

$$A_{\text{left joint}}(p) = \begin{cases} 1 - \sqrt{1 - A(\sqrt{x^2 + y^2})} & \text{if } x \leq 0 \\ 1 - \frac{1 - A(y)}{\sqrt{1 - A(\sqrt{x^2 + y^2})}} & \text{if } x > 0 \end{cases}$$

The right joint can be calculated with symmetry. This approximating formula is derived from defining an alpha density function and blend colors from infinite small segments on the polyline. But constrained by space, we won't dive into the details.

2.2 Filling and Gap Detection

We use the CGAL 2D Arrangements library [The CGAL Project 2023] for arrangement and point-location query, and CGAL Visibility library for computing visibility. We use the point-location query for bucket fill with a point and calculate the zone for bucket fill with a polyline. Filling a region can be roughly decomposed into three steps: construct the arrangement object, query faces, and render the polygon. As users draw strokes or deform strokes' shapes, arrangement object is updated accordingly. We query the faces with the user mouse position or stroke, and extract the polygons from the faces and render them with GPU. Please refer to our supplementary video for live demos.

REFERENCES

- Blender Online Community. 2023. *Blender - a 3D modelling and rendering package*. Blender Foundation, Stichting Blender Foundation, Amsterdam. <http://www.blender.org>
- Shen Cialo. 2023. *Ciallo*. <https://github.com/ShenCialo/Ciallo>
- Mark Dokter, Jozef Hladky, Mathias Parger, Dieter Schmalstieg, Hans-Peter Seidel, and Markus Steinberger. 2019. Hierarchical rasterization of curved primitives for vector graphics rendering on the GPU. In *Computer Graphics Forum*, Vol. 38. Wiley Online Library, 93–103.
- Mark J. Kilgard. 2020. Polar Stroking: New Theory and Methods for Stroking Paths. *ACM Trans. Graph.* 39, 4, Article 145 (aug 2020), 15 pages. <https://doi.org/10.1145/3386569.3392458>
- Amal Dev Parakkat, Pooran Memari, and Marie-Paule Cani. 2022. Delaunay Painting: Perceptual Image Colouring from Raster Contours with Gaps. *Computer Graphics Forum* 41, 6 (2022), 166–181. <https://doi.org/10.1111/cgf.14517> arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.14517>
- Aaron Rutten. 2022. 12 Ways to Fix BRUSH LAG - Digital Art Tutorial. <https://www.youtube.com/watch?v=e9KA16TadBg>
- The CGAL Project. 2023. *CGAL User and Reference Manual* (5.5 ed.). CGAL Editorial Board. <https://doc.cgal.org/5.5.1/Manual/packages.html>
- Chuan Yan, John Joon Young Chung, Yoon Kiheon, Yotam Gingold, Eytan Adar, and Sungsoo Ray Hong. 2022. FlatMagic: Improving Flat Colorization through AI-Driven Design for Digital Comic Professionals. , Article 380 (2022), 17 pages. <https://doi.org/10.1145/3491102.3502075>
- Jerry Yin, Chenxi Liu, Rebecca Lin, Nicholas Vining, Helge Rhodin, and Alla Sheffer. 2022. Detecting Viewer-Perceived Intended Vector Sketch Connectivity. *ACM Trans. Graph.* 41, 4, Article 87 (jul 2022), 11 pages. <https://doi.org/10.1145/3528223.3530097>